

IF2240 Basdat

Database Design using E-R Model

[Introduction](#)

[Design Phases](#)

[Design Alternatives](#)

[Design Approaches](#)

[E-R Model](#)

[Database Modeling](#)

[Entity Sets](#)

[Complex Attributes](#)

[Relationship Sets](#)

[Relationship Sets with Attributes](#)

[Roles](#)

[Degree of a Relationship Set](#)

[Binary Relationship](#)

[Non-Binary Relationship](#)

[Mapping Cardinality Constraints](#)

[One to One](#)

[One to Many](#)

[Many to One](#)

[Many to Many](#)

[Cardinality Constraints on Ternary Relationship](#)

[Total and Partial Participation](#)

[Weak Entity Sets](#)

[Specialization](#)

[Generalization](#)

[Design Constraints on a Specialization/Generalization](#)

[Notes:](#)

[Aggregation](#)

[Common Mistakes in E-R Diagrams](#)

[Redundant Attributes](#)

[Entities vs. Attributes](#)

[Entities vs. Relationship sets](#)

Introduction

Design Phases

1. Initial phase: characterize fully the data needs of the prospective database users
2. Second phase: choosing a data model
3. Final phase: moving from an abstract data model to the implementation of the database
 - a. Logical design – deciding on the database schema
 - b. Physical design – deciding on the physical layout of the database

Design Alternatives

In designing a database schema, we must ensure that we avoid two major pitfalls:

1. **Redundancy**: a bad design may result in repeat information and data inconsistency among the various copies of information
2. **Incompleteness**: a bad design may take certain aspects of the enterprise difficult or impossible to model

Design Approaches

1. Entity Relationship Model
 - a. Models an enterprise as a collection of entities and relationships
 - i. **Entity**: a “thing” or “object” in the enterprise that is distinguishable from other objects (Described by a set of attributes)
 - ii. **Relationship**: an association among several entities
 - iii. Represented by an entity-relationship diagram
2. Normalization Theory
 - a. Formalize what designs are bad, and test for them.

E-R Model

Database Modeling

The ER data model was developed to facilitate database design by allowing specification of an enterprise schema that represents the overall logical structure of a database.

→ hasil analisis, not hasil desain

The ER model employs three basic concepts:

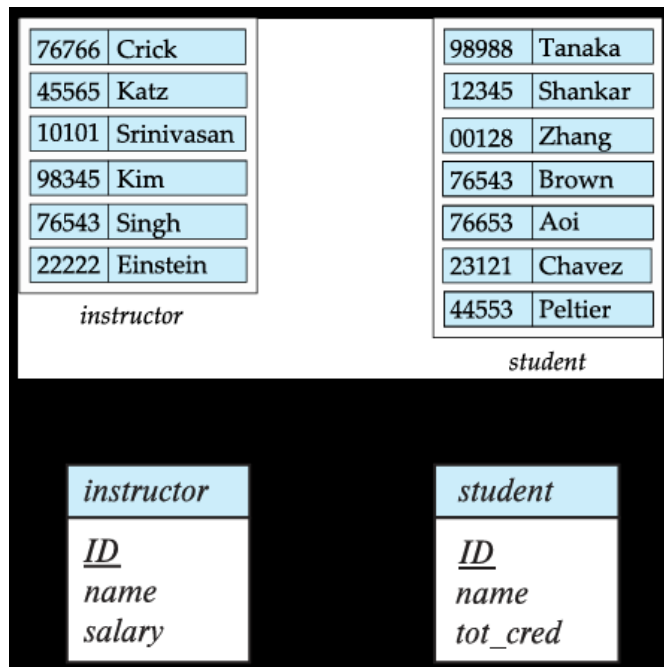
- entity sets

- relationship sets → tidak harus punya atribut, namun bisa punya
→ tidak boleh ada duplikat (asosiasi tidak boleh berulang")
- attributes

The ER model also has an associated diagrammatic representation, the ER diagram, which can express the overall structure of a database graphically.

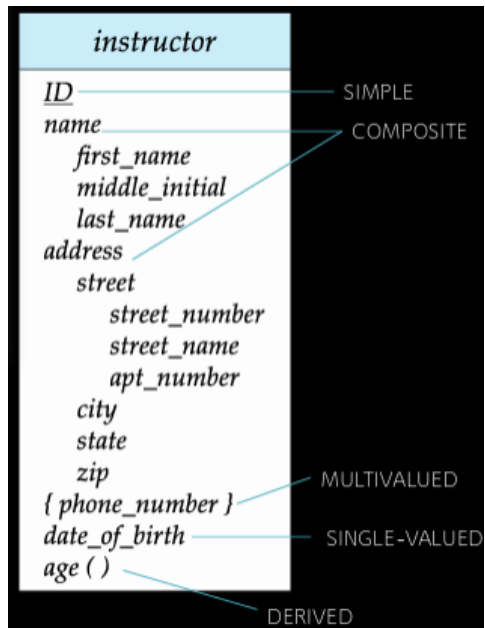
Entity Sets

- An entity is an object that exists and is distinguishable from other objects
- An entity set is a set of entities of the same type that share the same properties
- An entity is represented by a set of attributes
- Entity sets can be represented graphically as follows:
 - Rectangles represent entity sets



- Attributes listed inside entity rectangle
- Underline indicates primary key attributes

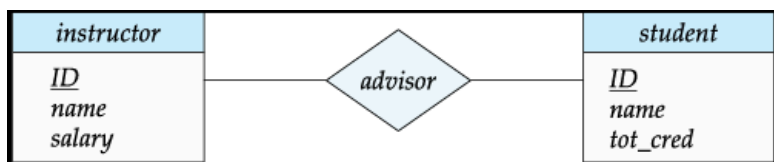
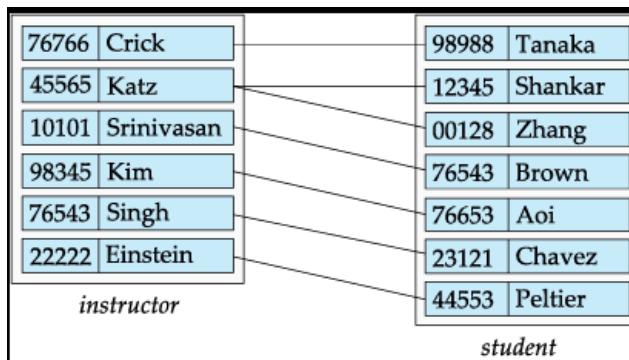
Complex Attributes



- Simple and composite
- Single-valued and multivalued (such as phone numbers, email)
- Derived (can be computed from other attributes), such as age and date of birth

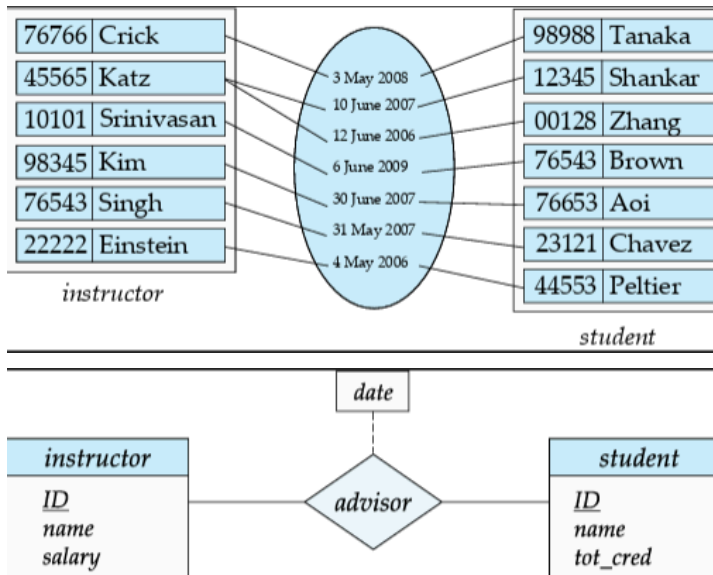
Relationship Sets

- A relationship is an association among several entities
- Diamond represent relationship sets
- Example: we define the relationship set advisor to denote the associations between students and the instructors who act as their advisors



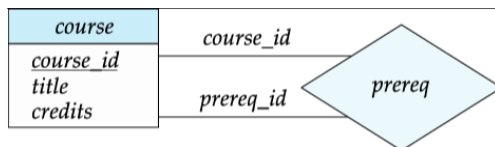
Relationship Sets with Attributes

- An attribute can also be associated with a relationship set.
- Example: The advisor relationship set between entity sets instructor and student may have the attribute date which tracks when the student started being associated with the advisor



Roles

- Entity sets of a relationship need not be distinct
- Example: the labels “course_id” and “prereq_id” are called roles



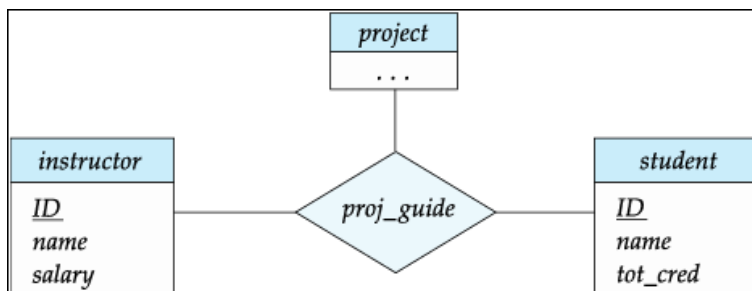
Degree of a Relationship Set

Binary Relationship

- Involve two entity sets
- Most relationship sets in a database system are binary

Non-Binary Relationship

- Relationships between more than two entity sets are rare
- Example: students work on research projects under the guidance of an instructor. Relationship *proj_guide* is a ternary relationship between instructor, student, and project

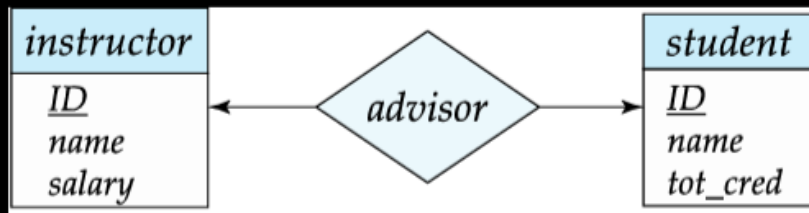


Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set
- For a binary set, the mapping cardinality must be one of the following types:
 - one to one
 - one to many
 - many to one
 - many to many

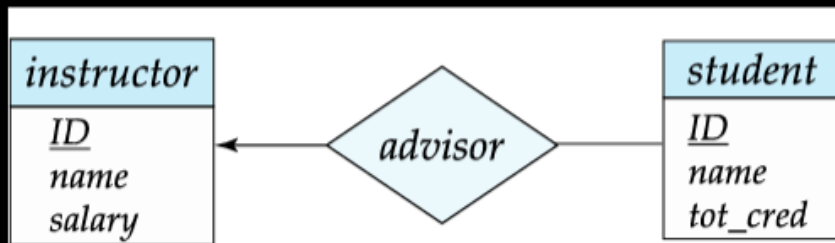
One to One

- A student is associated with at most one *instructor* via the relationship *advisor*
- A student is associated with at most one *department* via *stud_dept*



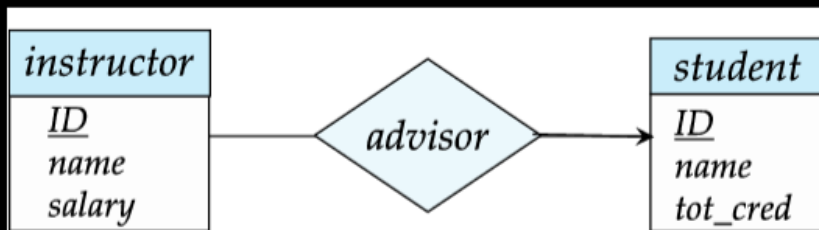
One to Many

- an instructor is associated with several (including 0) students via *advisor*
- a student is associated with at most one instructor via *advisor*,



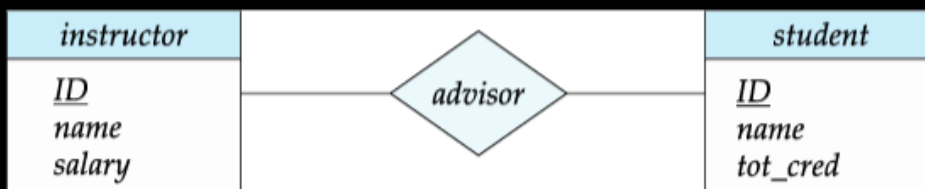
Many to One

- an *instructor* is associated with at most one student via *advisor*,
- and a student is associated with several (including 0) instructors via *advisor*

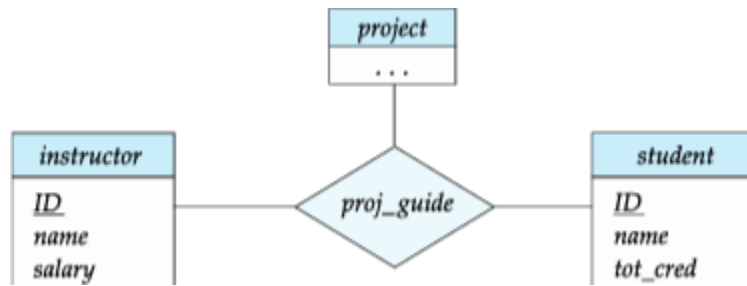


Many to Many

- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*,



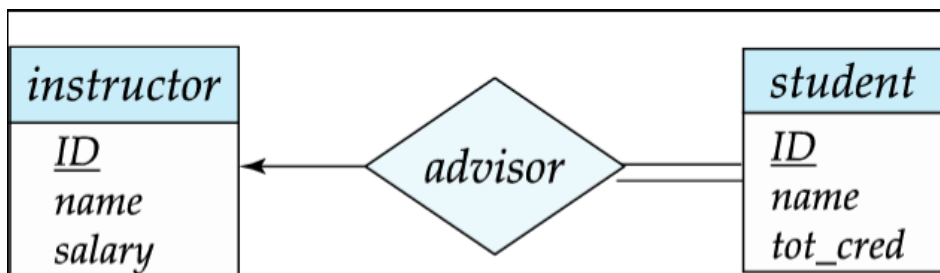
Cardinality Constraints on Ternary Relationship



- We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint
- For example, an arrow from proj_guide to instructor indicates each student has AT MOST ONE guide for a project
- If there is more than one arrow, there are two ways of defining the meaning:
 - For example, a ternary relationship R between A, B, C with arrows to B and C could mean:
 - Each A entity is associated with a unique entity from B and C **OR**
 - Each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B entity
 - To avoid confusion, we outlaw more than one arrow.

Total and Partial Participation

- **Total** (double line): every entity in the entity set participates in at least one relationship in the relationship set
- **Partial**: some entities may not participate in any relationship in the relationship set
- **Example**: participation of student in advisor relation is total because every student must have an associated instructor. Meanwhile, participation of instructor in advisor relation is partial because some instructor may not have an associated student.



Weak Entity Sets

- A weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond
- Example:

Consider a section entity, which is uniquely identified by a course_id, semester, year, and sec_id. Clearly, section entities are related to course entities. Suppose we create a relationship set sec_course between entity sets section and course. Note that the information in sec_course is redundant, since section already has an attribute course_id, which identifies the course with which the section is related. One option to deal with this redundancy is to get rid of the relationship sec_course; however, by doing so the relationship between section and course becomes implicit in an attribute, which is not desirable. An alternative way to deal with this redundancy is to **not store** the attribute **course_id** in the **section** entity and to only store the remaining attributes section_id, year, and semester. However, the entity set section then does not have enough attributes to identify a particular section entity uniquely

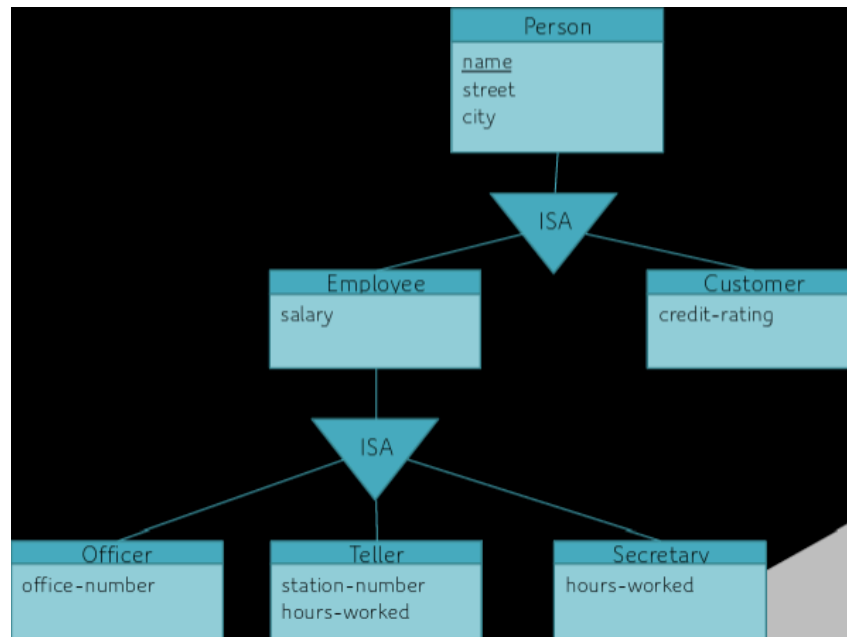
To deal with this problem, we treat the relationship sec_course as a special relationship that provides extra information, in this case, the **course_id**, **required to identify section** entities uniquely. → section entity set is dependent on course entity.



Primary key for section – (course_id, sec_id, semester, year) AKA discriminator, because it uniquely identify a weak entity

Note: untuk mengetahui kelas harus diketahui kode kelas, kode matkul, semester apa, tahun apa, tetapi di course juga ada course_id sehingga double jika section juga punya course_id (redundancy). Sehingga section membutuhkan atribut yang merupakan primary key dari entity set lain (yaitu course) → tanpa primary key course, section tidak dapat berdiri sendiri (thus its a weak entity set).

Specialization

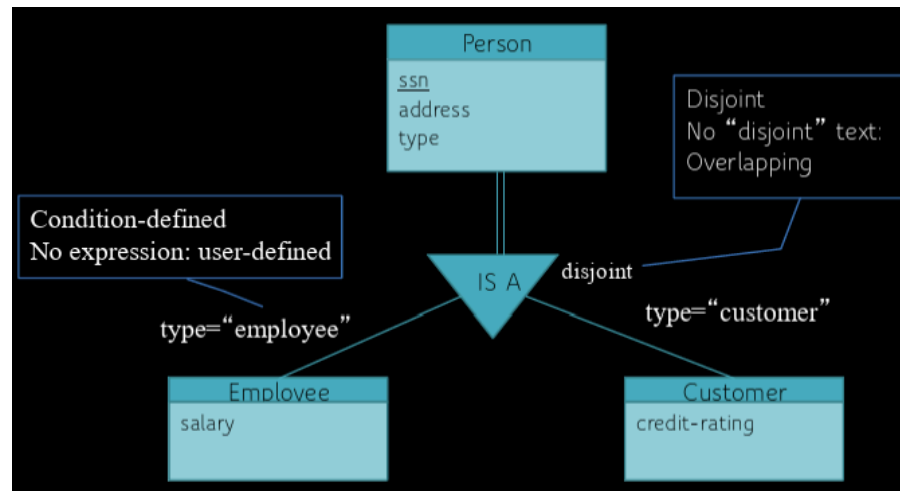


- Dari sebuah entity set yang umum, dapat dibuat sub-entity set yang lebih spesifik
- **Top-down design process:** we designate subgroupings within an entity set that are distinctive from other entities in the set
- Depicted by a **triangle** component labeled ISA (i.e. customer “is a” person)
- **Attribute inheritance:** a lower level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.

Generalization

- **Bottom-up design process:** combine a number of entity sets that share the same features into a higher-level entity set

Design Constraints on a Specialization/Generalization



- Constraint on which entities can be members of a given lower level entity set:
 - **condition-defined**
 - **user-defined**
- Constraint on whether or not entities may belong to more than one lower level entity set within a single generalization:
 - **disjoint** → an entity can belong to only one lower-level entity set
 - **overlapping** → an entity can belong to more than one lower level entity set
- Completeness constraint: specifies whether or not an entity in the higher level entity set must belong to at least one of the lower level entity sets within a generalization
 - **total**: an entity must belong to one of the lower level entity sets
 - **partial**: an entity need not belong to one of the lower level entity sets

Notes:

Kuis Kecil...

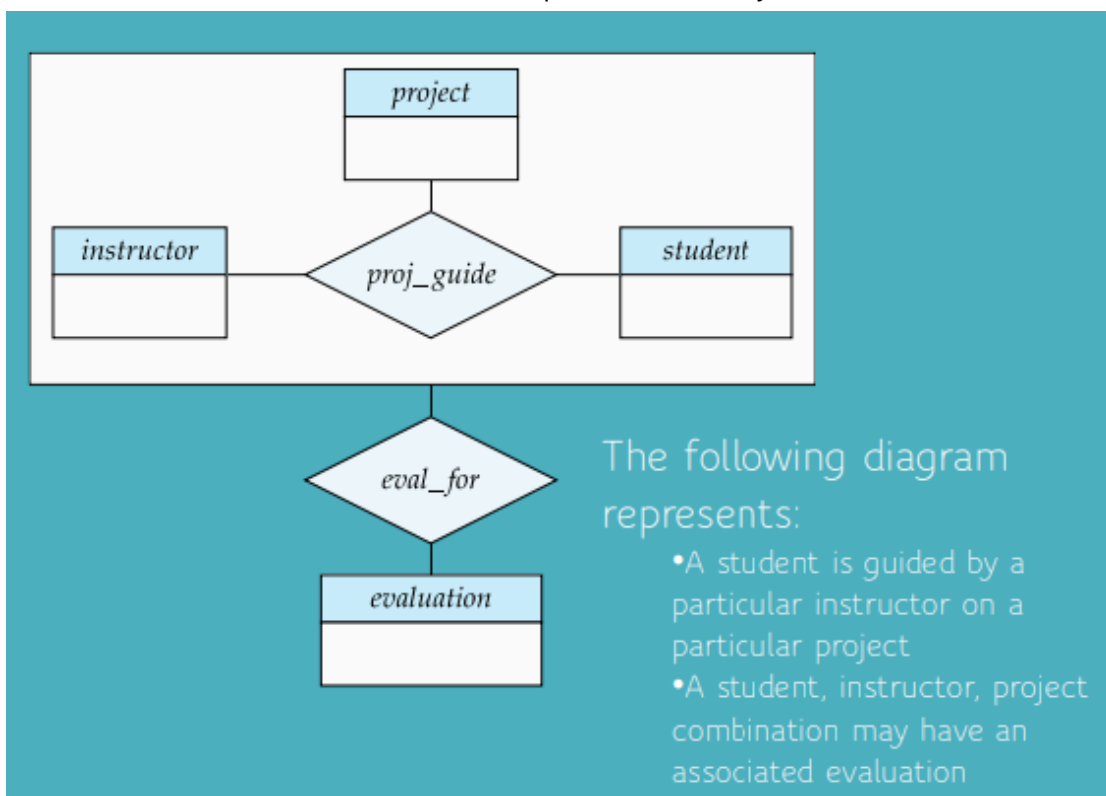
Apakah dibutuhkan spesialisasi/ generalisasi dalam persoalan ini?

Sebuah rental mobil mengklasifikasi kendaraan mereka menjadi empat katagori: compact, mid-size, full-size, dan sport utility. Tidak ada informasi khusus yang perlu disimpan untuk masing-masing katagori. Rental juga melakukan pencatatan data kendaraan berupa plat nomor (unik untuk setiap kendaraan), tahun pembuatan, model, tahun kepemilikan oleh rental, dan warna kendaraan. Seorang pelanggan dapat meminjam satu atau lebih kendaraan. Masing-masing pelanggan memiliki kode (unik untuk setiap pengguna), nama, alamat, dan email.

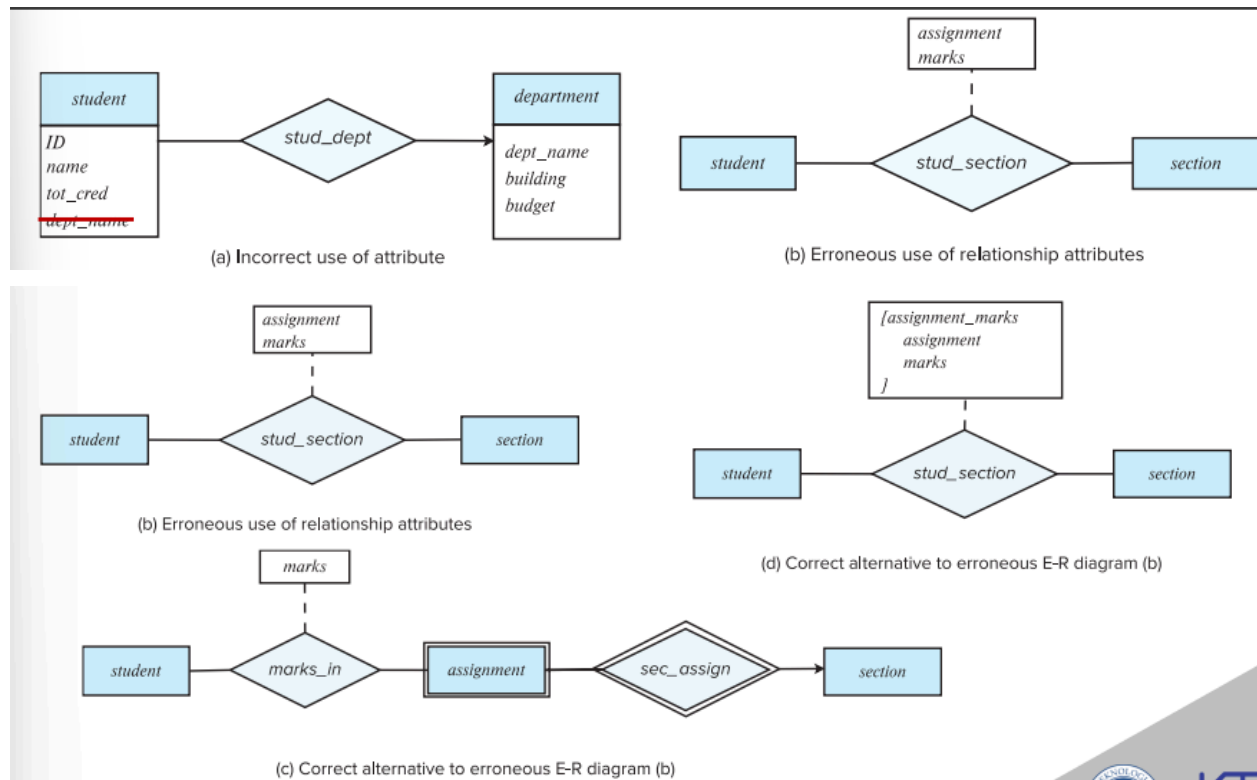
Tidak perlu spesialisasi/generalisasi, karena tidak ada karakteristik khusus yang diperlukan/digunakan.

Aggregation

- Menerapkan abstraksi terhadap sebuah relationship set agar dapat diperlakukan seperti entity set abstrak.
- Eliminate redundancy via aggregation:
 - treat relationship as an abstract entity
 - allows relationships between relationships
 - abstraction of relationship into new entity

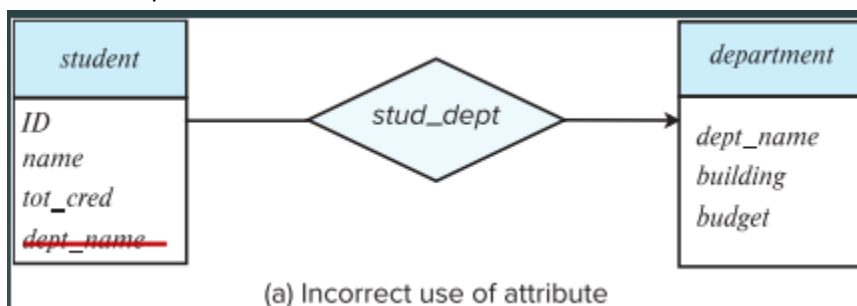


Common Mistakes in E-R Diagrams

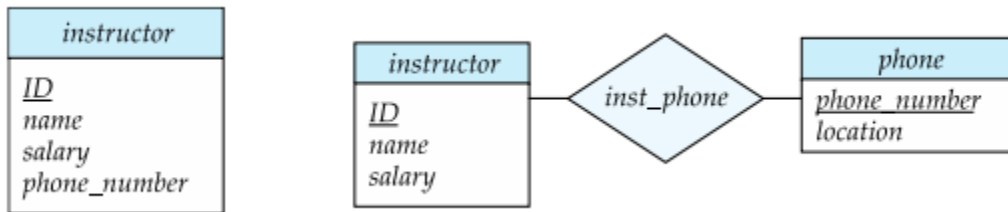


Redundant Attributes

- Suppose we have entity sets:
 - student, with attributes ID, name, tot_cred, dept_name
 - department, with attributes dept_name, building, budget
- We model the fact that each student has an associated department using a relationship set stud_dept
- The attribute dept_name in student below replicates information present in the relationship and is therefore **redundant** → needs to be removed



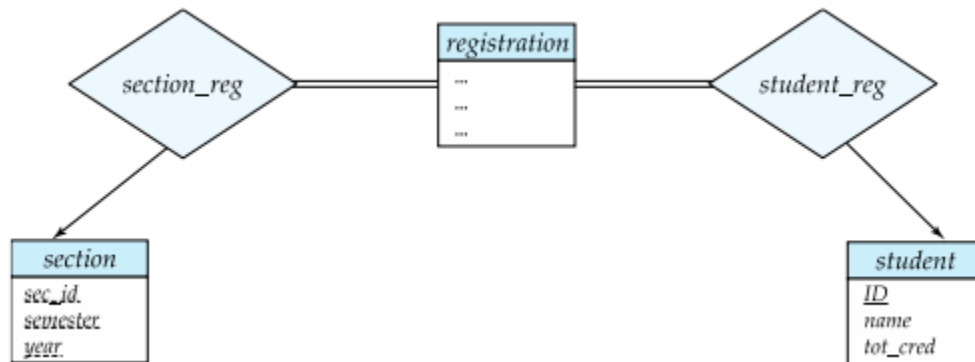
Entities vs. Attributes



Use of phone as an entity allows extra information about phone numbers (plus multiple phone numbers)

Entities vs. Relationship sets

- Use of entity sets vs. relationship sets



- Placement of relationship attributes:
For example, attribute date as attribute of advisor or as attribute of student